

판다스(Pandas) Series

객체 생성 및 사용법 강의 교재

광명코딩학원 | 파이썬 판다스 기초 강의

[핵심 개념] 판다스의 Series는 1차원 데이터를 저장하는 자료구조로, 데이터와 인덱스를 함께 저장하여 리스트와 딕셔너리의 장점을 모두 활용할 수 있습니다.

1. 파이썬 및 NumPy 자료구조 복습

자료구조	특징	기호	비유
리스트	순서 있음, 수정 가능	[]	기차 (중간에 타고 내릴 수 있음)
튜플	순서 있음, 수정 불가	()	비행기 (좌석 변경 불가)
딕셔너리	키 - 값 쌍, 레이블 인덱싱	{ }	사전 (단어:뜻 구조)
ndarray	다차원 배열, 벡터화 연산 가능	np.array()	엑셀 수식 자동 계산

2. 판다스(Pandas) 기본 자료구조

[개념] 판다스는 데이터 분석을 위한 파이썬 라이브러리입니다. numpy를 기반으로 만들어졌으며, Series와 DataFrame 두 가지 자료구조를 제공합니다.

자료구조	차원	설명	비유
Series	1차원	테이블의 컬럼 하나를 저장하는 자료구조	엑셀의 열(컬럼) 하나
DataFrame	2차원	여러 Series가 모인 테이블 형태의 자료구조	엑셀 시트 전체

3. 판다스 импорт 방법

임포트 방식

방법 1: from 형태 — 클래스를 직접 임포트

```
from pandas import Series, DataFrame
```

```
s = Series([100, 200, 300]) # Series 바로 사용
```

방법 2: as 축약 형태 — 가장 많이 사용

```
import pandas as pd
```

```
s = pd.Series([100, 200, 300]) # pd.Series 형태로 사용
```

방법	코드	사용 방식	특징
from 임포트	from pandas import Series, DataFrame	Series(...) 직접 사용	클래스명만 써도 됨
as 축약	import pandas as pd	pd.Series(...) 형태	가장 널리 쓰이는 방식

[TIP] 두 방식 모두 많이 사용됩니다. 참조하는 코드 방식에 맞춰 사용하세요. 이 교재에서는 import pandas as pd 방식을 기준으로 설명합니다.

4. Series 객체 생성 — 3가지 방법

방법 1. 리스트로 생성 (인덱스 자동 지정)

Series — 리스트

```
import pandas as pd

data = [100, 200, 300]

s = pd.Series(data)

print(s)

# 0 100
# 1 200
# 2 300

# dtype: int64
```

- 인덱스를 지정하지 않으면 0부터 시작하는 정수 인덱스 자동 부여
- 리스트뿐 아니라 튜플도 데이터로 사용 가능

방법 2. 리스트 + 인덱스 직접 지정

Series — 인덱스 지정

```
data = [100, 200, 300]

index = ['철수', '영희', '아름']

s = pd.Series(data, index)

print(s)

# 철수 100
# 영희 200
# 아름 300

# dtype: int64
```

- 데이터 개수와 인덱스 개수가 반드시 같아야 함 (다르면 ValueError)

- 인덱스는 문자열, 숫자, 날짜 등 다양하게 지정 가능

방법 3. 딕셔너리로 생성

Series — 딕셔너리

```
data_dict = {'철수': 100, '영희': 200, '아름': 300}

s = pd.Series(data_dict)

print(s)

# 철수 100

# 영희 200

# 아름 300

# dtype: int64

# 딕셔너리의 key -> 인덱스, value -> 데이터로 자동 변환
```

- 딕셔너리의 key가 자동으로 인덱스(레이블)가 됨
- 순서는 딕셔너리에 저장된 순서 그대로 유지

5. Series 내부 구조 및 인덱스 이해

Series 내부 구조:

행번호(숨김) | 인덱스(레이블) | 값(value)

- - - - - | - - - - - | - - - - -

0 | 철수 | 100

1 | 영희 | 200

2 | 아름 | 300

* 행 번호: 내부적으로 항상 존재 (눈에 보이지 않음)

* 인덱스: 직접 지정한 레이블 (출력 시 왼쪽에 표시)

* 값 : 실제 저장된 데이터 (출력 시 오른쪽에 표시)

[핵심] 인덱스를 지정하지 않으면 행번호와 인덱스가 같은 값(0,1,2...)이 됩니다. 인덱스를 지정하면 행번호와 인덱스가 다른 별개의 값이 됩니다.

접근 방법	언제 사용?	예시
인덱스(레이블)로 접근	인덱스 값을 알고 있을 때	s['철수'] - > 100
행 번호로 접근 (iloc)	위치(순서)로 접근할 때, 인덱스를 모를 때	s.iloc[- 1] - > 마지막 값
정수 인덱스(loc)	인덱스가 정수인 경우	s.loc[0] - > 첫 번째 값

6. Series 속성 접근 (.index / .values)

속성 접근 예제

```
import pandas as pd

data = [100, 200, 300]

index = ['철수', '영희', '아름']

s = pd.Series(data, index)

# 인덱스 확인

print(s.index)

# Index(['철수', '영희', '아름'], dtype='object')

# 값(데이터)만 확인

print(s.values)

# [100 200 300]

# 데이터 타입 확인

print(s.dtype) # int64
```

```
# 객체 타입 확인
```

```
print(type(s)) #
```

속성/메서드	설명	반환값 예시
s.index	인덱스(레이블) 반환	Index(['철수','영희','아름'])
s.values	데이터(값) ndarray 반환	[100 200 300]
s.dtype	데이터 타입 확인	int64
s.shape	배열 형태 확인	(3,)
s.name	Series 이름 확인/설정	None (미설정시)

7. Series 생성 실습 예제

예제 1: 아이스크림 가격 Series

아이스크림 가격 실습

```
import pandas as pd

data = [500, 800, 200]

index = ['메로나', '누가바', '빠빼코']

s = pd.Series(data, index)

print(s)

# 메로나 500

# 누가바 800

# 빠빼코 200

# dtype: int64

print(type(s)) #

print(s.index) # Index(['메로나', '누가바', '빠빼코'], dtype='object')

print(s.values) # [500 800 200]
```

예제 2: 주식 종가 Series

주식 종가 실습

```
import pandas as pd

data = [73000, 356000, 367000, 68600, 34150]

index = ['삼성전자', '셀트리온', '카카오', '삼성전자우', '현대바이오']

s = pd.Series(data, index)
```

```
print(s)

# 삼성전자 73000

# 셀트리온 356000

# 카카오 367000

# 삼성전자우 68600

# 현대바이오 34150

# dtype: int64
```

8. 시리즈와 딕셔너리

Series < - > 딕셔너리

```
import pandas as pd

data = [100, 200, 300]

index = ['철수', '영희', '아름']

s = pd.Series(data, index)

# Series - > 딕셔너리 변환

s_dict = s.to_dict()

print(s_dict)

# {'철수': 100, '영희': 200, '아름': 300}

# 딕셔너리 - > Series 변환

d = {'철수': 100, '영희': 200, '아름': 300}

s2 = pd.Series(d)
```

비교 항목	파이썬 딕셔너리	판다스 Series
인덱싱 방식	key로만 접근	key(레이블) + 행번호 모두 가능

비교 항목	파이썬 딕셔너리	판다스 Series
순서 보장	Python 3.7+ 보장	항상 보장
벡터화 연산	불가	가능 (numpy 기반)
변환	-	to_dict()로 딕셔너리 변환 가능

9. 시리즈와 엑셀 (저장 및 읽기)

엑셀 저장 / 읽기

```
import pandas as pd

data = [100, 200, 300]

index = ['철수', '영희', '아름']

s = pd.Series(data, index)

# Series -> 엑셀 파일로 저장

s.to_excel('data.xlsx')

# 엑셀에 저장: A열=인덱스, B열=값

# 엑셀 파일 읽어오기

df = pd.read_excel('data.xlsx', index_col=0)

series = df.iloc[:, 0] # 첫 번째 컬럼을 Series로

print(series)

# 철수 100

# 영희 200

# 아름 300

# Name: 0, dtype: int64
```

[TIP] s.to_excel('파일명.xlsx') 로 저장하면 인덱스가 A열, 값이 B열에 저장됩니다. 읽어올 때는 pd.read_excel()을 사용하며, index_col=0 옵션으로 인덱스 열을 지정합니다.

10. 주요 개념 정리

개념/속성	설명
Series	판다스의 1차원 자료구조. 인덱스(레이블) + 값(value) 쌍으로 저장
행 번호	내부적으로 항상 존재하는 0,1,2... 숫자 위치 (눈에 보이지 않음)
인덱스	각 데이터에 대응하는 레이블. 직접 지정하거나 자동(0,1,2) 부여
.index	Series의 인덱스(레이블) 반환
.values	Series의 값(데이터)를 ndarray로 반환
.dtype	저장된 데이터의 타입 확인 (int64, float64, object 등)
to_dict()	Series를 파이썬 딕셔너리로 변환
to_excel()	Series를 엑셀 파일로 저장
pd.read_excel()	엑셀 파일을 읽어와 DataFrame으로 반환

[실습] 실습 팁: 주피터 노트북에서 각 코드를 직접 실행해 보세요. 아나콘다를 사용하면 pandas가 이미 설치되어 있어 바로 사용 가능합니다.

판다스 Series 연습문제

총 10문제 | 광명코딩학원

Q1. 모듈 импорт

판다스의 Series 클래스를 사용하기 위해 주로 사용하는 두 가지 방식으로 импорт하세요.

힌트: `from pandas import ... / import pandas as pd`

내 답안:

Q2. 시리즈의 생성 (리스트)

다음 데이터를 시리즈 객체로 생성하세요.

점수
100
200
300

힌트: `pd.Series([100, 200, 300])`

내 답안:

Q3. 시리즈의 생성 (인덱스 지정)

생성된 시리즈 객체에 인덱스를 추가하세요.

인덱스	점수
철수	100
영희	200
아름	300

힌트: `pd.Series(data, index=['철수','영희','아름'])`

내 답안:

Q4. 시리즈의 생성 (딕셔너리)

다음 데이터를 파이썬 딕셔너리로 표현한 후, 딕셔너리를 사용하여 시리즈 객체를 생성하세요.

인덱스	점수
철수	100
영희	200
아름	300

힌트: `pd.Series({'철수':100, '영희':200, '아름':300})`

내 답안:

Q5. 시리즈 생성 — 주식 데이터

다음 네이버 금융 주식 데이터에서 종목명을 인덱스, 종가를 데이터로 하여 시리즈 객체를 생성하세요.

종목명	종가
삼성전자	73000
셀트리온	356000
카카오	367000
삼성전자우	68600
현대바이오	34150

힌트: `pd.Series(data, index=종목명리스트)`

내 답안:

Q6. 시리즈 인덱스

다음 시리즈에서 저장된 인덱스만 출력하세요.

```
data = [100, 200, 300]
```

```
index = ['철수', '영희', '아름']
```

```
s = pd.Series(data, index)
```

힌트: `s.index`

내 답안:

Q7. 시리즈 데이터

다음 시리즈에서 인덱스 없이 저장된 데이터(값)만 출력하세요.

```
data = [100, 200, 300]

index = ['철수', '영희', '아름']

s = pd.Series(data, index)

# 기대 결과: [100 200 300]
```

힌트: s.values

내 답안:

Q8. 시리즈와 딕셔너리

다음 시리즈를 딕셔너리로 변환하세요.

```
data = [100, 200, 300]

index = ['철수', '영희', '아름']

s = pd.Series(data, index)

# 기대 결과: {'철수': 100, '영희': 200, '아름': 300}
```

힌트: s.to_dict()

내 답안:

Q9. 시리즈와 엑셀 (저장)

다음 시리즈를 엑셀 파일(data.xlsx)에 저장하세요.

```
data = [100, 200, 300]

index = ['철수', '영희', '아름']

s = pd.Series(data, index)
```

힌트: s.to_excel('data.xlsx')

내 답안:

Q10. 시리즈와 엑셀 (읽기)

저장된 엑셀 파일(data.xlsx)을 읽어 오세요. 아래 힌트 함수를 활용하세요.

힌트 함수: pd.read_excel

기대 결과:

철수 100

영희 200

아름 300

Name: 0, dtype: int64

힌트: pd.read_excel('data.xlsx', index_col=0).iloc[:, 0]

내 답안:

정답 모음

판다스 Series 연습문제 정답 | 광명코딩학원

Q1. 모듈 임포트	<code>from pandas import Series, DataFrame / import pandas as pd</code>
Q2. 시리즈 생성 (리스트)	<code>s = pd.Series([100, 200, 300])</code>
Q3. 시리즈 생성 (인덱스)	<code>s = pd.Series([100,200,300], index=['철수','영희','아름'])</code>
Q4. 시리즈 생성 (딕셔너리)	<code>d = {'철수':100, '영희':200, '아름':300} / s = pd.Series(d)</code>
Q5. 시리즈 생성 (주식)	<code>data=[73000,356000,367000,68600,34150] / index=['삼성전자','셀트리온','카카오','삼성전자우','현대바이오'] / s = pd.Series(data, index)</code>
Q6. 시리즈 인덱스	<code>print(s.index) / # Index(['철수', '영희', '아름'], dtype='object')</code>
Q7. 시리즈 데이터	<code>print(s.values) # [100 200 300]</code>
Q8. 시리즈 - > 딕셔너리	<code>print(s.to_dict()) / # {'철수': 100, '영희': 200, '아름': 300}</code>
Q9. 시리즈 - > 엑셀 저장	<code>s.to_excel('data.xlsx')</code>
Q10. 엑셀 - > 시리즈 읽기	<code>df = pd.read_excel('data.xlsx', index_col=0) / series = df.iloc[:, 0]</code>

[완료] 수고하셨습니다! Series의 생성부터 엑셀 저장까지 모두 학습했습니다. 다음 시간에는 DataFrame에 대해 배워보겠습니다.